

Personal Developer Learning Portal

Product Requirements & Milestone-Based Development Roadmap

Product type: Personal interactive developer-learning platform **Primary purpose:** Guide one learner from beginner JavaScript through full-stack software engineering using a structured, milestone-based learning methodology.

1. Product Vision

Build a personal learning portal that turns the Full-Stack JavaScript Developer Roadmap into an **interactive apprenticeship system**.

The application must not simply present courses or documentation.

It must actively guide the learner through:

```
Roadmap
↓
Concept
↓
Explanation
↓
Examples
↓
Practice
↓
Exercises
↓
Quiz
↓
Coding Challenge
↓
Mini Project
↓
Review
↓
Knowledge Assessment
↓
Milestone Gate
↓
Next Concept
```

The system should continuously answer:

What should I learn next?

What should I practice?

Do I actually understand this?

Where am I weak?

What should I review?

Am I ready to move forward?

Can I actually build something using what I learned?

2. Core Product Philosophy

The application must enforce the learning principles of the roadmap.

Principle 1 — Fundamentals before frameworks

The platform must prevent premature progression.

Example:

```
JavaScript Fundamentals
  ↓
Runtime + Async
  ↓
DOM + Browser
  ↓
HTTP
  ↓
Node
  ↓
Database
  ↓
Express
  ↓
Authentication
  ↓
React
```

The learner should not be encouraged to jump directly into React while foundational JavaScript concepts remain weak.

3. Principle 2 — Understanding over memorization

The system must distinguish between:

```
"I have seen this concept."
```

and

```
"I understand and can use this concept."
```

Therefore, each concept needs multiple forms of assessment.

Example:

Concept: Array `.map()`

The learner should encounter:

1. Explanation
2. Example
3. Predict-the-output question
4. Multiple-choice question

5. Code-completion exercise
6. Coding exercise
7. Debugging exercise
8. Real-world usage question
9. Mini challenge
10. Review assessment

4. Principle 3 — Build instead of merely watch

Every major section must eventually produce something tangible.

For example:

```
Functions
  ↓
Exercises
  ↓
Mini challenge
  ↓
Small application
```

The platform should minimize passive learning.

5. Principle 4 — Debugging is part of learning

The application must deliberately introduce broken code.

Example:

```
const users = ["John", "Mary", "Peter"];

const names = users.map(user => {
  user.name;
});
```

The learner should be asked:

Why doesn't this produce the expected result?

The system should evaluate the learner's reasoning, not simply provide the answer.

6. Principle 5 — AI is a mentor, not an answer machine

AI assistance must be designed around learning.

The system should prefer:

```
Hint
↓
Question
↓
Explanation
↓
Smaller hint
↓
Example
↓
Solution
```

rather than immediately giving:

```
Complete solution
```

The learner should be encouraged to attempt the problem first.

7. Principle 6 — Progression must be evidence-based

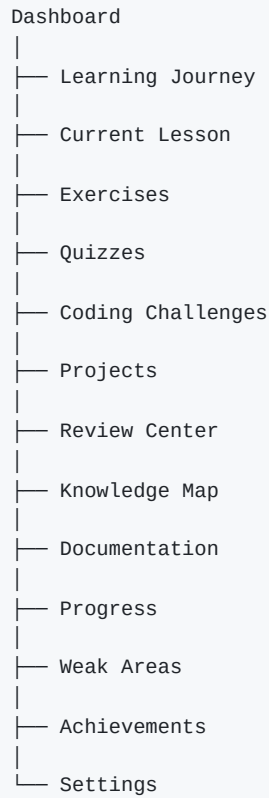
Completing a lesson does not automatically mean mastery.

Progress should be determined from:

```
Exercises
+
Quiz results
+
Coding challenges
+
Projects
+
Review performance
+
Retention
```

8. High-Level Application Structure

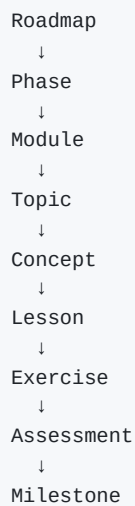
The application should contain:



- Dashboard
- Learning Journey
- Current Lesson
- Exercises
- Quizzes
- Coding Challenges
- Projects
- Review Center
- Knowledge Map
- Documentation
- Progress
- Weak Areas
- Achievements
- Settings

9. Core Learning Model

The curriculum should be hierarchical.



```
graph TD; Roadmap --> Phase; Phase --> Module; Module --> Topic; Topic --> Concept; Concept --> Lesson; Lesson --> Exercise; Exercise --> Assessment; Assessment --> Milestone;
```

Roadmap

↓

Phase

↓

Module

↓

Topic

↓

Concept

↓

Lesson

↓

Exercise

↓

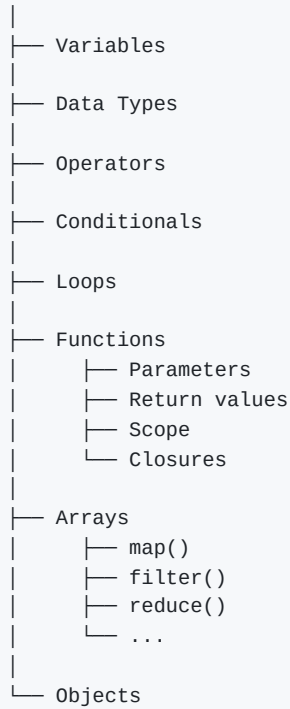
Assessment

↓

Milestone

Example:

Phase 2 – JavaScript Core



10. Milestone-Based Development Plan

The development team should build the application incrementally.

Do NOT attempt to build the entire platform at once.

MILESTONE 0 — Product Foundation

Goal

Establish the product architecture and learning model before building UI features.

Tasks

- ☐ Define product requirements
- ☐ Define roadmap hierarchy
- ☐ Define learning states
- ☐ Define mastery states
- ☐ Define exercise types
- ☐ Define assessment types
- ☐ Define milestone rules
- ☐ Define user progress model
- ☐ Define content model
- ☐ Define database architecture
- ☐ Define application architecture
- ☐ Define API conventions

- ☐ Define coding standards
- ☐ Define testing strategy

Deliverables

Product specification
Database schema
API specification
Learning-state specification
Milestone specification
Content specification
UI wireframes
Technical architecture

Review Gate

The team must be able to answer:

| What exactly does the system consider "learning"?

| What exactly does it consider "mastery"?

| What conditions allow a learner to progress?

MILESTONE 1 — Application Shell

Goal

Create the basic learning portal.

Build

- ☐ Application layout
- ☐ Navigation
- ☐ Dashboard
- ☐ Sidebar
- ☐ Roadmap page
- ☐ Profile page
- ☐ Progress page
- ☐ Responsive layout
- ☐ Theme system
- ☐ Loading states
- ☐ Error states

Dashboard should show

Current Phase
Current Module
Current Lesson
Progress
Weak Areas
Upcoming Review
Current Milestone
Recommended Next Action

Example:

CURRENTLY LEARNING

JavaScript Core
→ Functions
→ Function Parameters

Progress
 80%

Today's work

- ☐ Read concept
- ☐ Complete 5 exercises
- ☐ Pass quiz
- ☐ Complete coding challenge

Review tomorrow
→ Variables
→ Scope

Review Gate

The learner can open the application and immediately understand:

Where am I and what should I do next?

MILESTONE 2 — Roadmap Engine

Goal

Turn the roadmap into structured application data.

Tasks

- ☐ Create Phase model
- ☐ Create Module model
- ☐ Create Topic model
- ☐ Create Concept model
- ☐ Create Lesson model
- ☐ Create Milestone model
- ☐ Build roadmap navigation

- ☐ Display prerequisite relationships
- ☐ Display progress
- ☐ Lock/unlock content based on prerequisites

Example

```
Phase 2
↓
JavaScript Core
↓
Functions
↓
Function Parameters
↓
Callbacks
```

The system should know:

```
Callbacks
requires:
Functions
Parameters
Arguments
Return values
Higher-order functions
```

Review Gate

The roadmap can be navigated entirely through the application without hard-coded page-to-page progression.

MILESTONE 3 — Lesson Engine

Goal

Create the system that actually teaches concepts.

Every lesson should follow a consistent structure.

```
Concept introduction
↓
Explanation
↓
Example
↓
Interactive example
↓
Try it yourself
↓
Practice
↓
Knowledge check
```

Lesson features

- ☐ Rich text
- ☐ Code blocks
- ☐ Syntax highlighting
- ☐ Diagrams
- ☐ Interactive examples
- ☐ "Explain it simply"
- ☐ Key concepts
- ☐ Common mistakes
- ☐ Related concepts
- ☐ Documentation references
- ☐ Completion tracking

Important

A lesson should not be marked mastered simply because the learner opened it.

Possible states:

```
Not started
Started
Learning
Practicing
Assessed
Mastered
Needs review
```

MILESTONE 4 — Exercise Engine

Goal

Build the core practice system.

Support multiple exercise types.

Exercise types

Multiple Choice

```
Which value does this expression return?
```

True / False

```
Array.map() mutates the original array.
```

Predict the Output

```
console.log(typeof null);
```

Fill in the Blank

```
const result = numbers.____(n => n * 2);
```

Code Ordering

Arrange code into the correct execution order.

Matching

Match concepts with definitions.

Debugging

Find and explain the bug.

Short Answer

Explain a concept in your own words.

Coding Exercise

Write executable JavaScript.

MILESTONE 5 — Code Execution Engine

Goal

Allow the learner to write and execute code safely.

Example:

Code Editor

```
function add(a, b) {  
  return a + b;  
}
```

[Run Code]

Output:

5

Tests:

- ✓ adds two numbers
- ✓ handles zero
- ✓ handles negative numbers

Requirements

- ☐ Code editor
- ☐ Syntax highlighting
- ☐ Run code
- ☐ Capture stdout
- ☐ Capture errors
- ☐ Timeout execution
- ☐ Memory/resource limits
- ☐ Hidden tests
- ☐ Public tests
- ☐ Test result display
- ☐ Secure sandboxing

Critical security requirement

Never execute arbitrary learner code directly inside the main application server.

Use an isolated execution environment.

Possible architecture:



The exact sandbox technology can be selected during implementation.

MILESTONE 6 — Quiz Engine

Goal

Create formal knowledge assessments.

Quiz features

- ☐ Question bank
- ☐ Randomized questions
- ☐ Difficulty levels
- ☐ Timed quizzes
- ☐ Untimed quizzes
- ☐ Topic-specific quizzes
- ☐ Phase quizzes
- ☐ Mixed quizzes
- ☐ Instant feedback
- ☐ Detailed explanations
- ☐ Score calculation
- ☐ Historical scores
- ☐ Retake functionality

Difficulty

Beginner
Intermediate
Advanced
Expert

But difficulty should eventually be based on actual performance rather than simply a manually assigned label.

MILESTONE 7 — Mastery Engine

This is one of the most important parts of the entire system.

The platform must determine whether the learner actually knows something.

Concept mastery score

Each concept receives a mastery score.

Example:

Functions

Knowledge	92%
Recall	85%
Application	78%
Debugging	65%
Retention	88%

Overall Mastery



The system should not rely solely on quiz scores.

Consider:

- Quiz performance
- Exercise performance
- Coding performance
- Debugging performance
- Repeated performance
- Time since last practice
- Number of hints
- Number of failed attempts

MILESTONE 8 — Review / Spaced Practice Engine

Goal

Prevent the learner from forgetting previously learned material.

The system should periodically bring older concepts back.

Example:

Today

New:

→ Functions

Review:

→ Variables

→ Operators

→ Arrays

Tomorrow:

New:

→ Scope

Review:

→ Functions

→ Arrays

→ Conditionals

Eventually implement spaced repetition.

A concept can have:

Next review:
Tomorrow

Then:
3 days

Then:
7 days

Then:
14 days

Then:
30 days

The exact algorithm can evolve later.

MILESTONE 9 — Adaptive Learning Engine

Now the application becomes genuinely personalized.

The system should detect:

Strong concepts
Weak concepts
Forgotten concepts
Frequently missed question types
Frequently used hints
Slow problem areas
Repeated mistakes

Example:

Your strongest areas:

- ✓ Variables
- ✓ Arrays
- ✓ Functions

Needs reinforcement:

- ▲ Closures
- ▲ `Array.reduce()`
- ▲ Scope

Recommended:

1. Review lexical scope
2. Complete 3 closure exercises
3. Debug 2 closure problems
4. Retake scope assessment

MILESTONE 10 — Documentation / Reference Center

Create an integrated documentation area.

The portal should provide references relevant to the current topic.

Example:

Currently learning:

JavaScript → Promises

Recommended references:

- MDN Promise
- JavaScript.info Promises
- MDN async/await
- Node.js asynchronous programming

The application should distinguish:

Learn
Practice
Reference

Documentation is not the same thing as the lesson.

MILESTONE 11 — Milestone Review Gates

Every major phase must have a formal gate.

Example:

JavaScript Core Gate

Knowledge

- ☐ Variables
- ☐ Types
- ☐ Operators
- ☐ Conditions
- ☐ Loops
- ☐ Functions
- ☐ Arrays
- ☐ Objects
- ☐ Scope
- ☐ Closures
- ☐ Modules

Practical

- ☐ Solve unseen problems
- ☐ Debug code
- ☐ Write functions
- ☐ Manipulate data
- ☐ Explain concepts

Project

Build a vanilla JavaScript application without following a tutorial.

Final Assessment

Knowledge test
+
Coding challenge
+
Debugging challenge
+
Project
+
Oral/written explanation

Only then:

PASS

Otherwise:

REINFORCE

MILESTONE 12 — Project Engine

Goal

Move from isolated exercises to real software development.

Every phase should contain projects.

Example:

```
JavaScript
↓
Todo Manager

DOM
↓
Dashboard

HTTP
↓
Weather/API application

Node
↓
CLI application

Express
↓
REST API

React
↓
Frontend application

Full Stack
↓
Task management system
```

Projects should have:

```
Requirements
Acceptance criteria
Suggested architecture
Constraints
Milestones
Tests
Submission
Review
```

MILESTONE 13 — Project Review System

After completing a project, the system should evaluate:

```
Functionality
Code quality
Architecture
Error handling
Accessibility
Security
Testing
Git usage
Documentation
```

The learner should receive feedback such as:

FUNCTIONALITY

9/10

CODE QUALITY

7/10

ARCHITECTURE

6/10

DEBUGGING

8/10

DOCUMENTATION

5/10

Overall

7.2/10

But the score should not be the only output.

The system should say:

Your next improvement should be error handling because you repeatedly allow API failures to reach the UI without controlled handling.

MILESTONE 14 — Git / Engineering Workflow

The learning platform should eventually teach professional workflow.

Projects should encourage:

Create repository

↓

Create branch

↓

Implement feature

↓

Commit

↓

Test

↓

Review

↓

Merge

Teach meaningful commits:

feat: add task creation

fix: handle empty task titles

test: add task validation tests

refactor: extract task service

docs: update API documentation

MILESTONE 15 — Debugging Lab

Create a dedicated debugging environment.

The system presents:

```
Broken application
+
Error
+
Expected behaviour
```

The learner investigates.

Example:

```
BUG #042

The task list is empty after refreshing the page.

Expected:
Tasks should persist.

Your job:
Find the root cause.
```

The learner should use:

```
Console
Network
Debugger
Logs
Code inspection
```

The system should evaluate the solution.

MILESTONE 16 — AI Mentor

Only after the foundational learning system works should AI become deeply integrated.

AI mentor capabilities:

```
Explain concept
Give hint
Ask Socratic questions
Generate additional exercises
Review explanation
Review code
Explain error
Suggest debugging strategy
Generate practice problems
Conduct mock interview
Run oral knowledge checks
```

AI should have levels of assistance

Level 0

No help

Level 1

Concept hint

Level 2

Directional hint

Level 3

Detailed explanation

Level 4

Example

Level 5

Solution

The system should track excessive assistance.

Example:

You solved 8/10 problems,
but used full solutions on 6.

Recommendation:

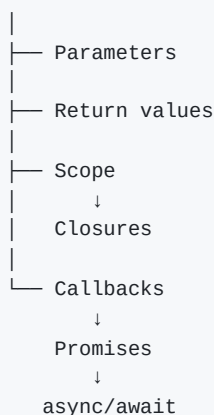
Repeat this topic without solutions.

MILESTONE 17 — Knowledge Graph

Represent relationships between concepts.

Example:

Functions



This allows the application to understand:

If the learner struggles with Promises, check whether Functions, callbacks and asynchronous execution are strong enough.

MILESTONE 18 — Personal Learning Dashboard

The dashboard should eventually become the learner's command center.

Display:

- Current phase
- Current topic
- Today's learning
- Review queue
- Mastery
- Weak areas
- Learning streak
- Projects
- Milestones
- Recent performance
- Upcoming reviews

Example:

YOUR DEVELOPER JOURNEY

Phase 2 – JavaScript Core

Progress

72%

Mastery

64%

TODAY

✓ Variables review

→ Functions lesson

→ 5 exercises

→ Coding challenge

NEEDS ATTENTION

▲ Closures

▲ Array.reduce()

NEXT MILESTONE

JavaScript Core Assessment

MILESTONE 19 — Full Roadmap Content

Once the platform works technically, populate the entire roadmap.

Phase 0

Developer environment

Phase 1

HTML + CSS

Phase 2

JavaScript Core

Phase 3

Runtime + Async

Phase 4

DOM + Browser

Phase 5

HTTP + REST

Phase 6

Node.js

Phase 7

SQL + PostgreSQL

Phase 8

Express + APIs

Phase 9

Authentication + Security

Phase 10

React

Phase 11

Full Stack

Phase 12

TypeScript

Phase 13

Testing

Phase 14

Production Engineering

Phase 15

Next.js

Phase 16

Advanced Engineering

MILESTONE 20 — Production Readiness

Before considering the platform complete, verify:

Learning

- ☐ Every roadmap phase exists
- ☐ Concepts have prerequisites
- ☐ Lessons exist
- ☐ Exercises exist
- ☐ Quizzes exist
- ☐ Coding challenges exist
- ☐ Projects exist
- ☐ Review gates exist

Assessment

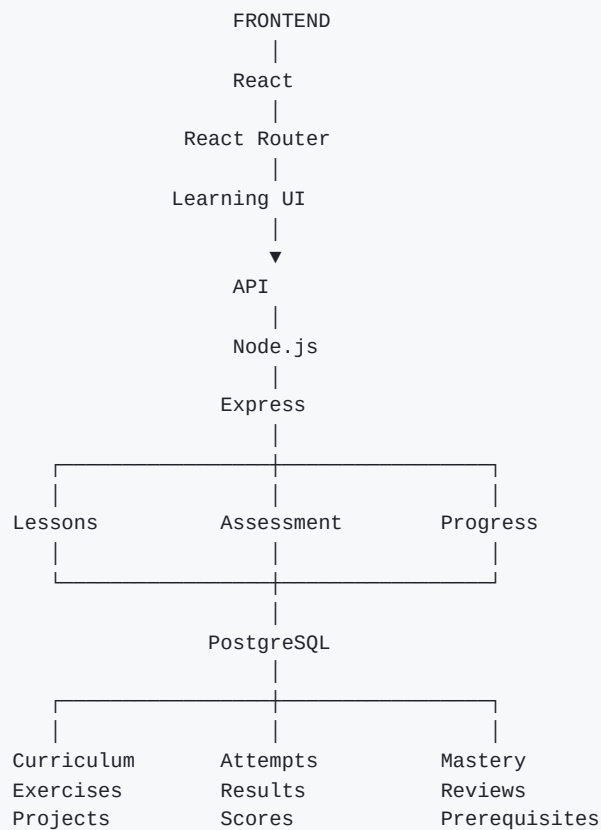
- ☐ Mastery tracking
- ☐ Spaced repetition
- ☐ Adaptive recommendations
- ☐ Weak-area detection
- ☐ Project evaluation
- ☐ Coding evaluation

Engineering

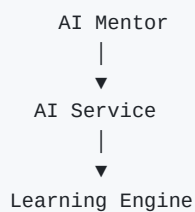
- ☐ Authentication
- ☐ Database
- ☐ API
- ☐ Testing
- ☐ Logging
- ☐ Error handling
- ☐ Security
- ☐ Deployment
- ☐ Backups
- ☐ Monitoring

11. Suggested Application Architecture

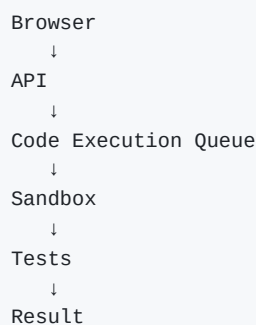
For this particular project, the eventual architecture could be:



Later:



And:



12. Core Database Entities

The team should initially model something similar to:

```
users
phases
modules
topics
concepts
lessons
prerequisites
exercises
questions
question_options
coding_challenges
projects
milestones
milestone_requirements
user_progress
concept_mastery
exercise_attempts
quiz_attempts
coding_attempts
project_submissions
reviews
review_schedule
hints
achievements
documentation_links
```

Don't over-engineer the initial schema.

Start with the minimum required for the first learning loop.

13. First MVP

The team should NOT attempt to build everything above initially.

The first usable version should be:

```
Login
↓
Dashboard
↓
Roadmap
↓
Phase
↓
Topic
↓
Lesson
↓
Exercise
↓
Quiz
↓
Score
↓
Progress
```

The MVP should support approximately:

Phase 0
+
first part of JavaScript fundamentals

For example:

Variables
Data types
Operators
Conditionals
Loops
Functions

The system should be fully functional for these concepts before expanding the curriculum.

14. MVP Acceptance Test

The development team should be able to demonstrate:

Scenario

A new learner signs in.

1. Dashboard opens.
2. System says "Start Phase 0."
3. Learner opens the first lesson.
4. Learner reads the concept.
5. Learner completes an exercise.
6. Learner gets feedback.
7. Learner takes a quiz.
8. System calculates performance.
9. Weak concepts are identified.
10. System schedules review.
11. Learner continues.
12. At the end of the module, a milestone assessment appears.
13. Learner passes.
14. Next module unlocks.

If this entire loop works, **we have the foundation of the product.**

15. Learning State Machine

The system should use explicit learning states.

```

LOCKED
↓
AVAILABLE
↓
STARTED
↓
LEARNING
↓
PRACTICING
↓
ASSESSED
↓
MASTERED

```

Failure path:

```

ASSESSED
|
├── PASS → MASTERED
|
└── FAIL
    ↓
    REINFORCE
    ↓
    PRACTICE
    ↓
    REASSESS

```

Review path:

```

MASTERED
↓
REVIEW DUE
↓
REVIEW
|
├── PASS → MASTERED
|
└── FAIL → REINFORCE

```

16. Definition of Mastery

A concept should not become "mastered" because the learner got one quiz right.

Initial mastery rules could require:

```

Quiz ≥ 80%
+
Exercises ≥ 80%
+
Coding challenge passed
+
No critical prerequisite weaknesses

```

For major milestones:

```
Knowledge assessment
+
Coding assessment
+
Debugging assessment
+
Project
```

Thresholds should be configurable rather than hard-coded.

17. Content Authoring System

The development team should make learning content data-driven.

Do NOT hard-code every lesson into React components.

Instead:

```
Database / Content files
      ↓
Learning Engine
      ↓
React UI
```

This allows new concepts to be added without rewriting application code.

An administrator/content author should eventually be able to create:

```
Phase
Module
Topic
Concept
Lesson
Exercise
Quiz
Challenge
Project
Documentation link
Milestone
```

18. Content Quality Requirements

Every concept should contain:

1. Definition
2. Why it matters
3. Mental model
4. Syntax
5. Examples
6. Common mistakes
7. Practice
8. Debugging challenge
9. Real-world use
10. Documentation references
11. Assessment
12. Prerequisites

Example:

Concept:
JavaScript Closures

Prerequisites:
Functions
Scope
Lexical scope

Learn:
Explanation

Understand:
Mental model

Practice:
Exercises

Apply:
Coding challenge

Debug:
Broken closure example

Assess:
Quiz

Prove:
Mini project

Reference:
MDN
JavaScript.info

Master:
Only after review gate

19. Developer Team Priorities

The team should prioritize in this order:

1. Learning model
2. Curriculum engine
3. Exercise engine
4. Assessment engine
5. Progress tracking
6. Review/mastery system
7. Coding environment
8. Projects
9. AI mentor
10. Advanced personalization

Do NOT prioritize:

Fancy animations
Gamification
Leaderboards
Social features
Avatars
Badges

before the learning engine works.

20. Gamification — Later

Gamification can eventually include:

XP
Levels
Achievements
Streaks
Milestones
Badges
Completion statistics

But it should reward **learning behaviour**, not merely time spent.

Good:

Solved difficult debugging challenge
Completed milestone
Recovered a weak concept
Built project independently

Bad:

Opened the app
Watched a lesson
Clicked "complete"

21. The Application's Most Important Feature

The most important feature is not the dashboard.

It is not the quiz engine.

It is not the AI.

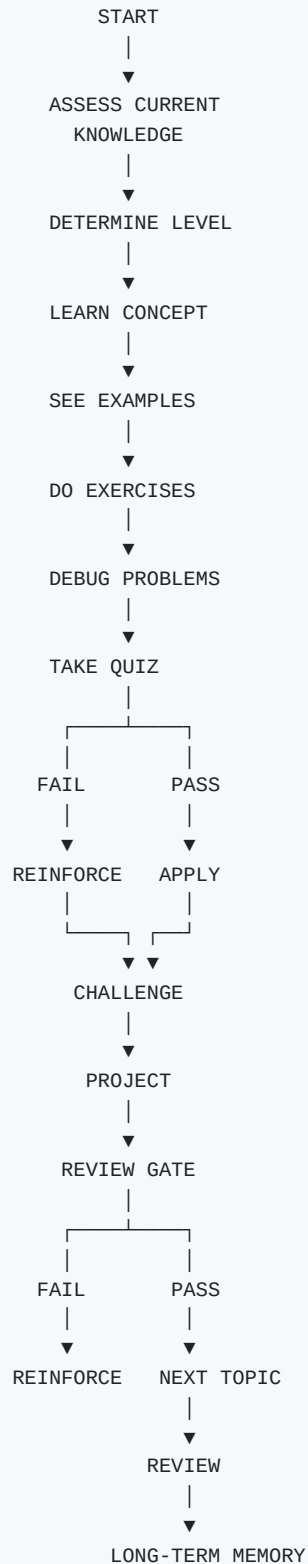
It is this:

The system must know what the learner knows, what the learner doesn't know, and what the learner should do next.

Everything else supports that objective.

22. Final Product Flow

The finished application should eventually feel like this:



23. Ultimate Goal

The application should eventually transform the roadmap from a static document:

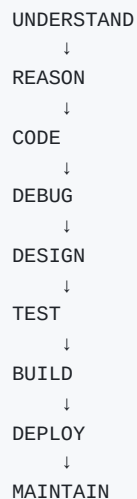
"Learn JavaScript → Learn Node → Learn React"

into an actual **personal engineering apprenticeship**:



The end result should be a system that doesn't simply help me **finish a JavaScript course**.

It should help me become a developer who can:



That is the standard against which the product should be developed.

24. Development Rule

Every feature added to the platform should be evaluated against one question:

Does this feature make the learner better at understanding, reasoning about, building, debugging, or engineering software?

If the answer is no, it should not be a priority.

The learning engine comes first.